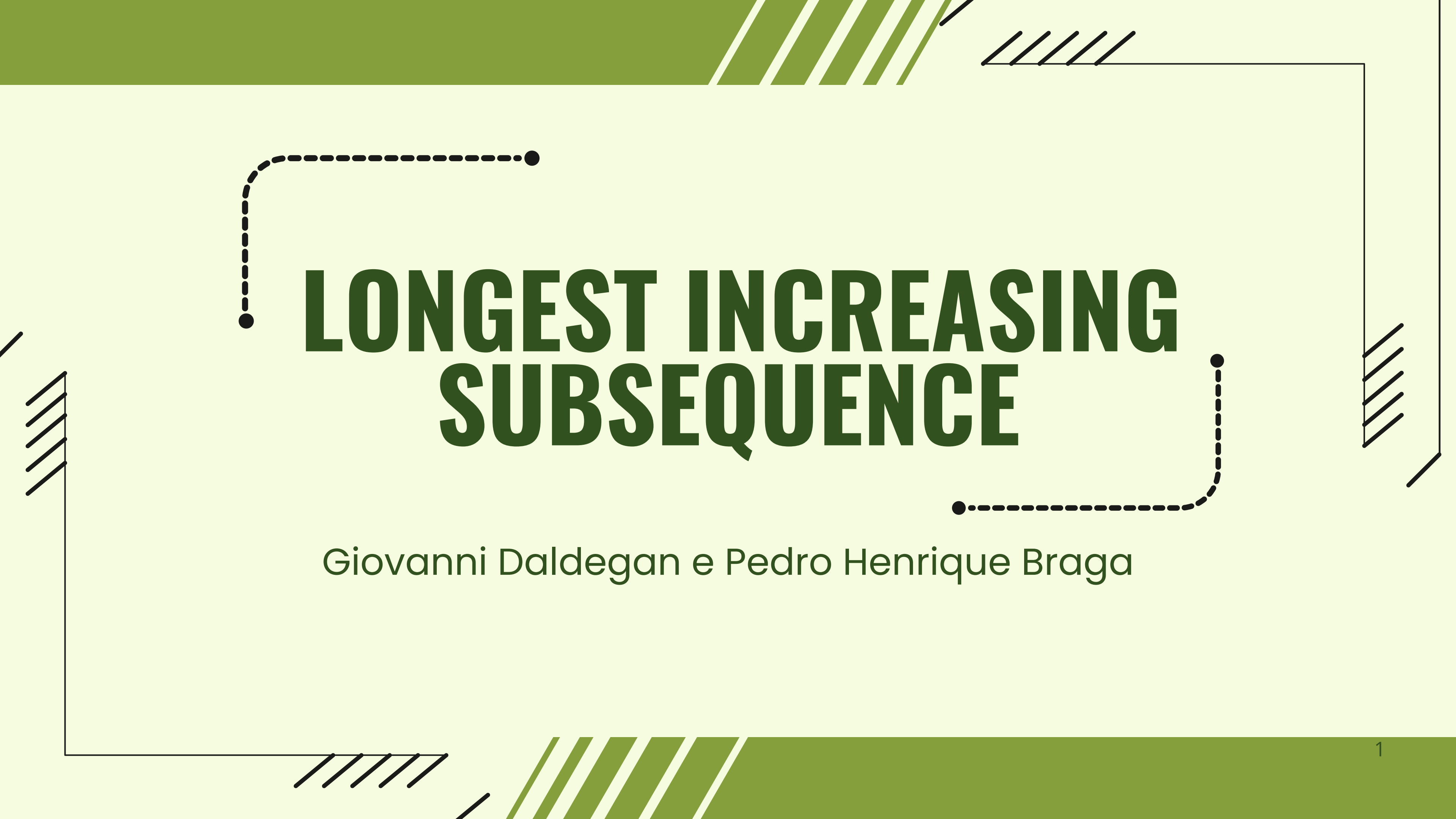




LONGEST INCREASING SUBSEQUENCE

Giovanni Daldegan e Pedro Henrique Braga

CSES - INCREASING SUBSEQUENCE

Time limit: 1.00 s **Memory limit:** 512 MB

You are given an array containing n integers. Your task is to determine the longest increasing subsequence in the array, i.e., the longest subsequence where every element is larger than the previous one.

A subsequence is a sequence that can be derived from the array by deleting some elements without changing the order of the remaining elements.

Input

The first line contains an integer n : the size of the array.

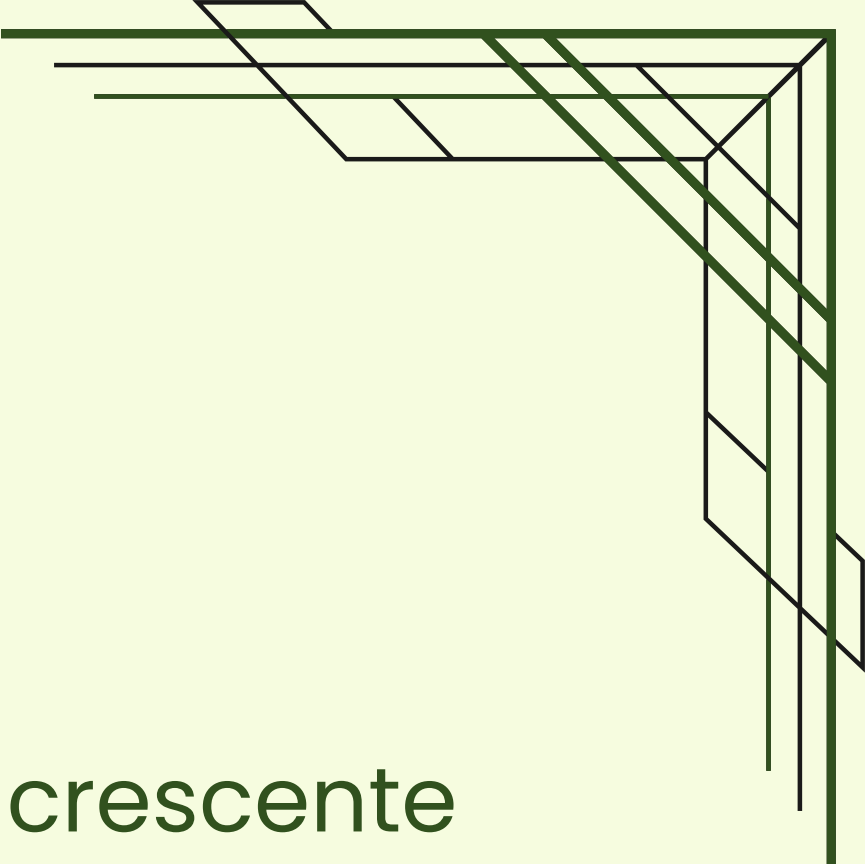
After this there are n integers $x_1, x_2, \dots, x_n$: the contents of the array.

Output

Print the length of the longest increasing subsequence.

Constraints

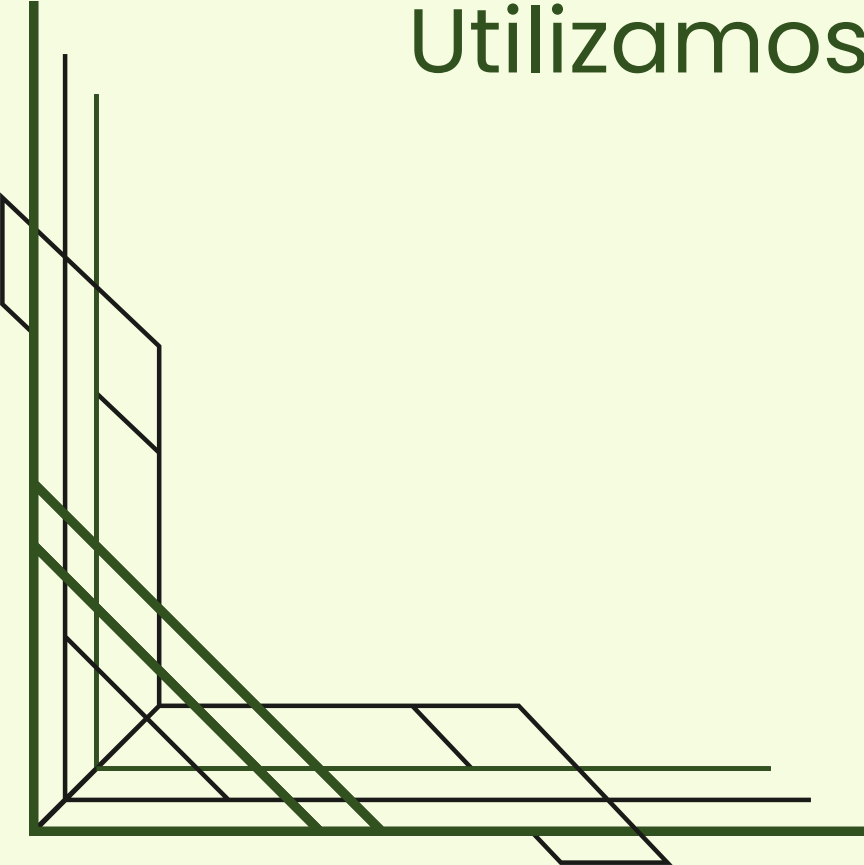
- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq x_i \leq 10^9$



SOLUÇÃO TRIVIAL

Vamos calcular o tamanho máximo da subsequência crescente terminada em cada uma das posições do array. A maior delas será a resposta do problema.

Utilizamos aqui o paradigma de programação dinâmica.



SOLUÇÃO TRIVIAL

Variáveis

- array: vetor com valores de entrada;
- LIS: vetor com contagem da maior subsequência terminada no elemento correspondente de array, inicializado em 1;
- longest: contagem da maior subsequência encontrada.

SOLUÇÃO TRIVIAL

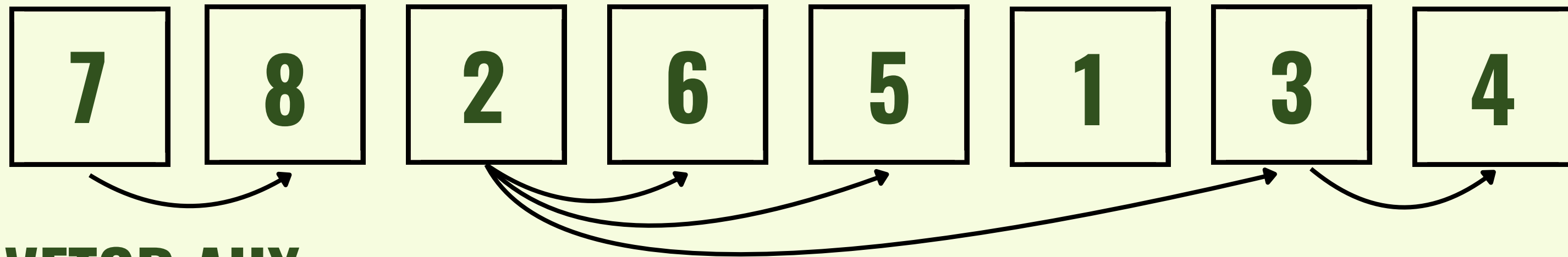
Algoritmo:

Para cada posição i do vetor, percorremos todas as posições j anteriores e checamos se elas podem fazer parte da maior subsequência terminada em $\text{array}[i]$.

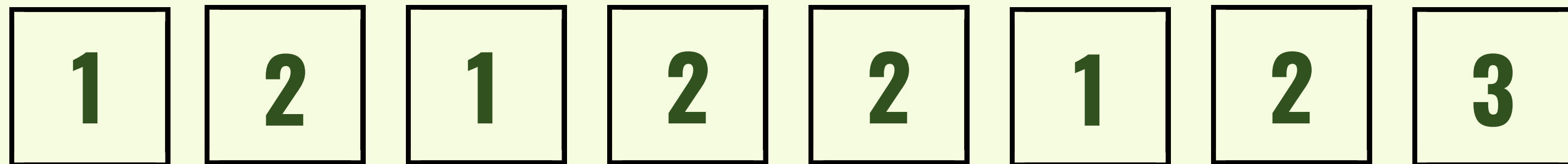
Se sim, atualizamos $\text{LIS}[i]$ para $\text{LIS}[j] + 1$. Caso contrário, mantemos $\text{LIS}[i]$ sem alterações.

SOLUÇÃO TRIVIAL

VETOR ORIGINAL:



VETOR AUX:



CÓDIGO E VEREDITO CSES

```
1 #include <bits/stdc++.h>
2 #define ll long long
3
4 using namespace std;
5
6 int main()
7 {
8     int size;
9     cin >> size;
10    vector<ll> array (size), LIS (size,1);
11    int longest = (size > 0);
12
13    for (int i = 0; i < size; i++) {
14        cin >> array[i];
15
16        for (int j = 0; j < i; j++) {
17            if (array[i] > array[j]) {
18                LIS[i] = max(LIS[i], LIS[j]+1);
19
20                if (longest < LIS[i])
21                    longest = LIS[i];
22            }
23        }
24    }
25    cout << longest << endl;
26    return 0;
27 }
```

test	verdict	time	
#1	ACCEPTED	0.00 s	»»
#2	ACCEPTED	0.00 s	»»
#3	ACCEPTED	0.00 s	»»
#4	ACCEPTED	0.00 s	»»
#5	ACCEPTED	0.00 s	»»
#6	TIME LIMIT EXCEEDED	--	»»
#7	TIME LIMIT EXCEEDED	--	»»
#8	TIME LIMIT EXCEEDED	--	»»
#9	TIME LIMIT EXCEEDED	--	»»
#10	TIME LIMIT EXCEEDED	--	»»
#11	ACCEPTED	0.00 s	»»
#12	ACCEPTED	0.00 s	»»
#13	TIME LIMIT EXCEEDED	--	»»
#14	ACCEPTED	0.00 s	»»
#15	ACCEPTED	0.00 s	»»
#16	ACCEPTED	0.00 s	»»
#17	ACCEPTED	0.00 s	»»
#18	TIME LIMIT EXCEEDED	--	»»
#19	ACCEPTED	0.00 s	»»



SOLUÇÃO OTIMIZADA

O solução utilizará a ideia de guardar o menor valor final para cada comprimento de subsequência encontrado até o momento e utilizará essa informação para processar os elementos do array.

Paradigma de programação empregado: Programação Dinâmica

Isso será feito utilizando uma adaptação no algoritmo de sorting de elementos de um array chamado *Patience Sorting*.





PATIENCE SORTING (ADAPTADO)

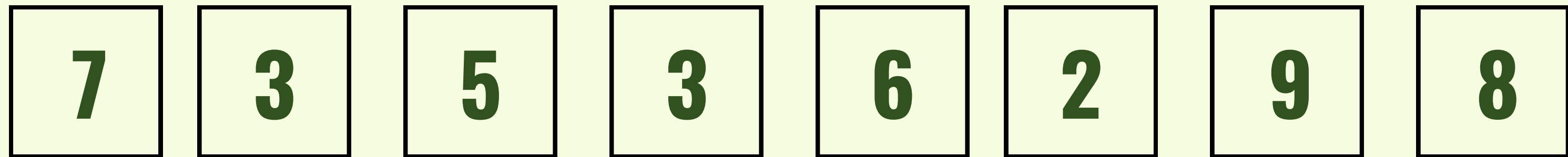
- Algoritmo de sorting baseado no jogo de cartas paciência.
- Criaremos um vetor auxiliar que armazenará as “tails” (finais) de cada pilha.
- Inseriremos em ordem os elementos do array no vetor auxiliar seguindo as seguintes regras:
 1. Na primeira pilha, na qual o elemento seja menor ou igual ao final da pilha, o substituímos pelo elemento;
 2. Se o elemento foi maior que todos os finais das pilhas cria-se uma nova pilha para esse elemento;



VISUALIZAÇÃO DA IMPLEMENTAÇÃO

Caso teste CSES:

VETOR ORIGINAL:



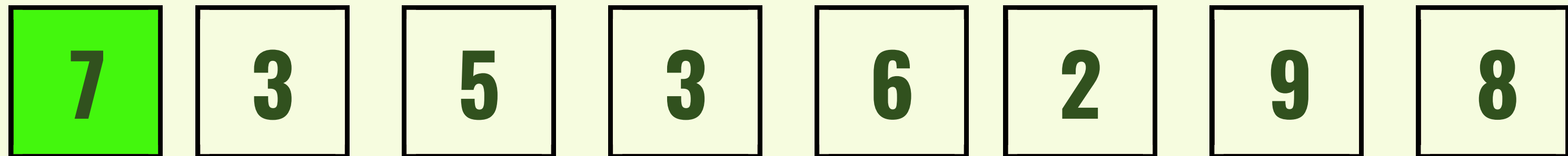
VETOR AUX:

(Inicialmente vazio)

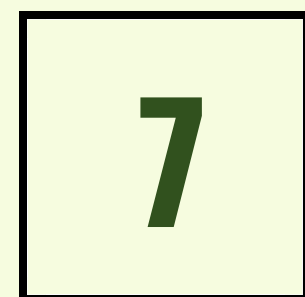
VISUALIZAÇÃO DA IMPLEMENTAÇÃO

Caso teste CSES:

VETOR ORIGINAL:



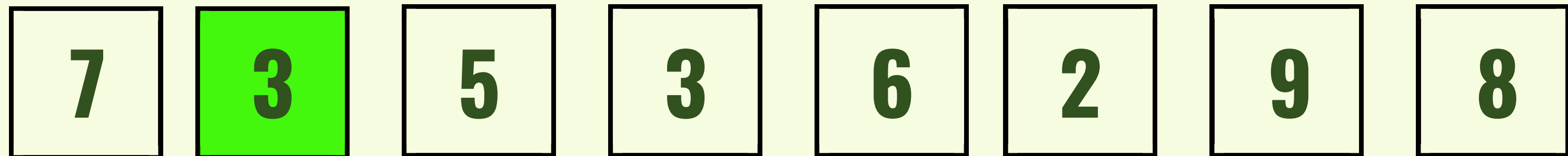
VETOR AUX:



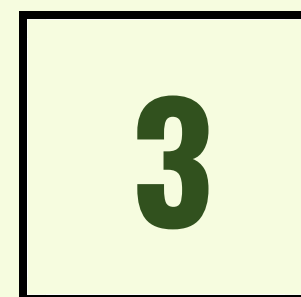
VISUALIZAÇÃO DA IMPLEMENTAÇÃO

Caso teste CSES:

VETOR ORIGINAL:



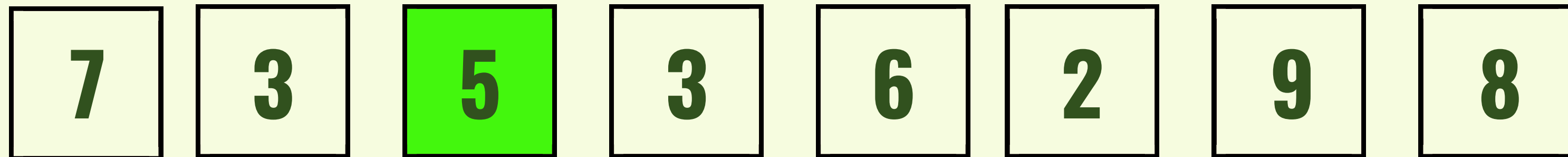
VETOR AUX:



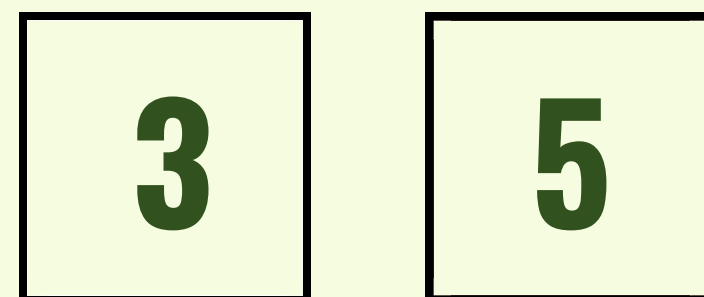
VISUALIZAÇÃO DA IMPLEMENTAÇÃO

Caso teste CSES:

VETOR ORIGINAL:



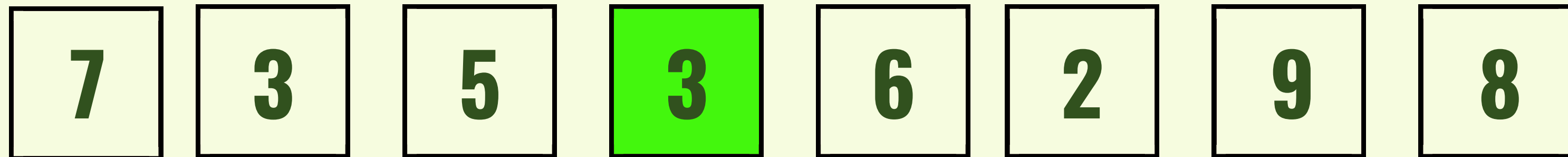
VETOR AUX:



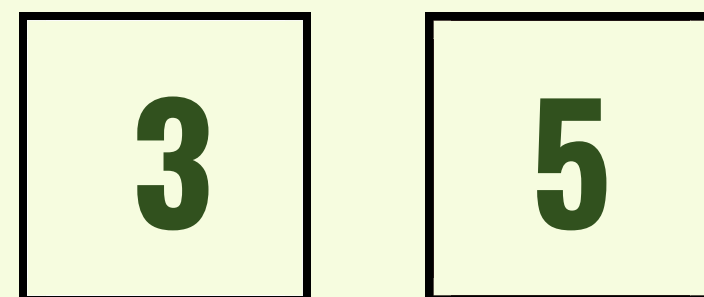
VISUALIZAÇÃO DA IMPLEMENTAÇÃO

Caso teste CSES:

VETOR ORIGINAL:



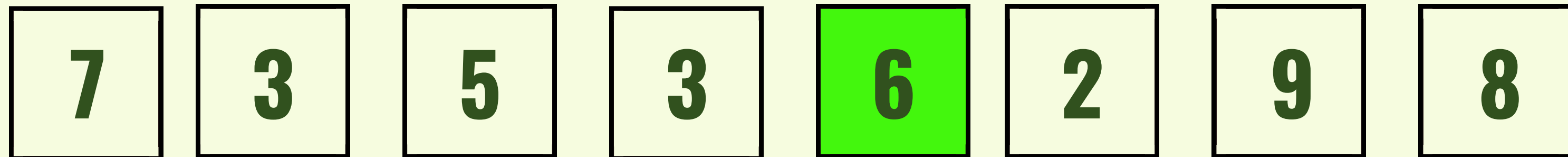
VETOR AUX:



VISUALIZAÇÃO DA IMPLEMENTAÇÃO

Caso teste CSES:

VETOR ORIGINAL:



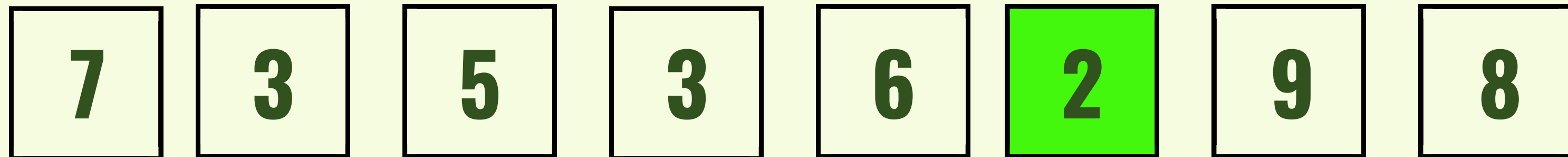
VETOR AUX:



VISUALIZAÇÃO DA IMPLEMENTAÇÃO

Caso teste CSES:

VETOR ORIGINAL:



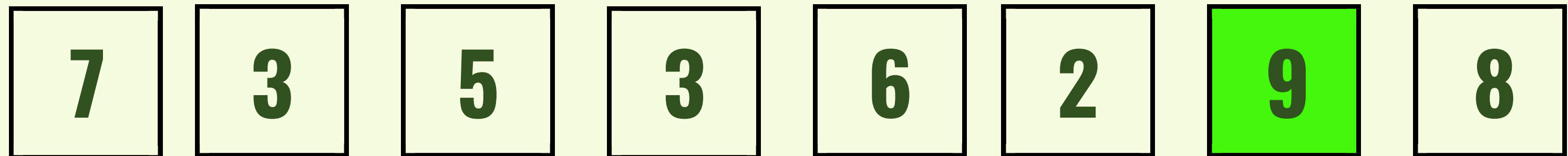
VETOR AUX:



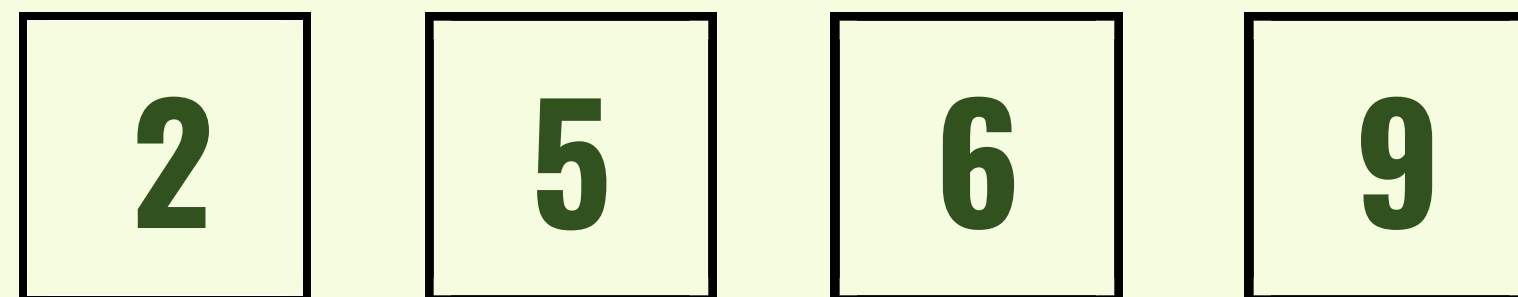
VISUALIZAÇÃO DA IMPLEMENTAÇÃO

Caso teste CSES:

VETOR ORIGINAL:



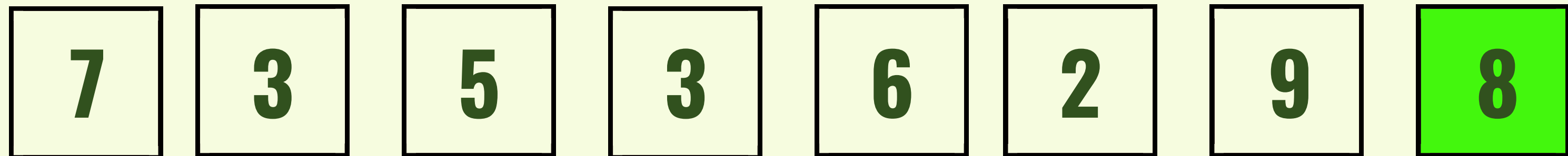
VETOR AUX:



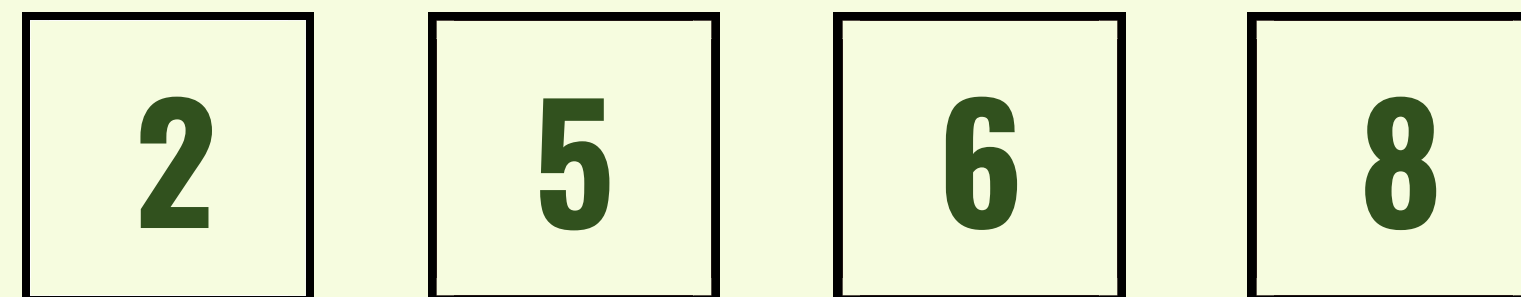
VISUALIZAÇÃO DA IMPLEMENTAÇÃO

Caso teste CSES:

VETOR ORIGINAL:



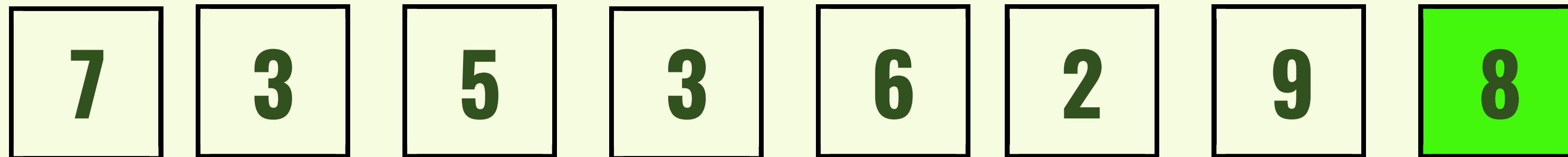
VETOR AUX:



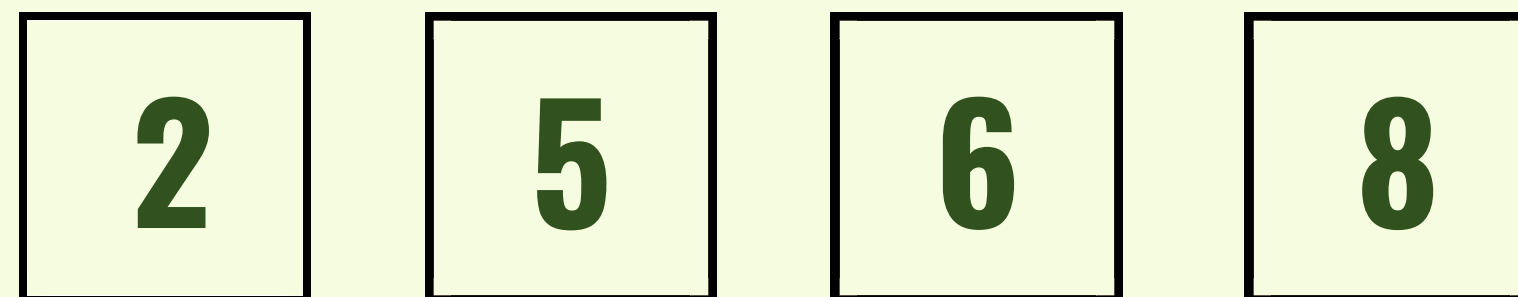
VISUALIZAÇÃO DA IMPLEMENTAÇÃO

Caso teste CSES:

VETOR ORIGINAL:



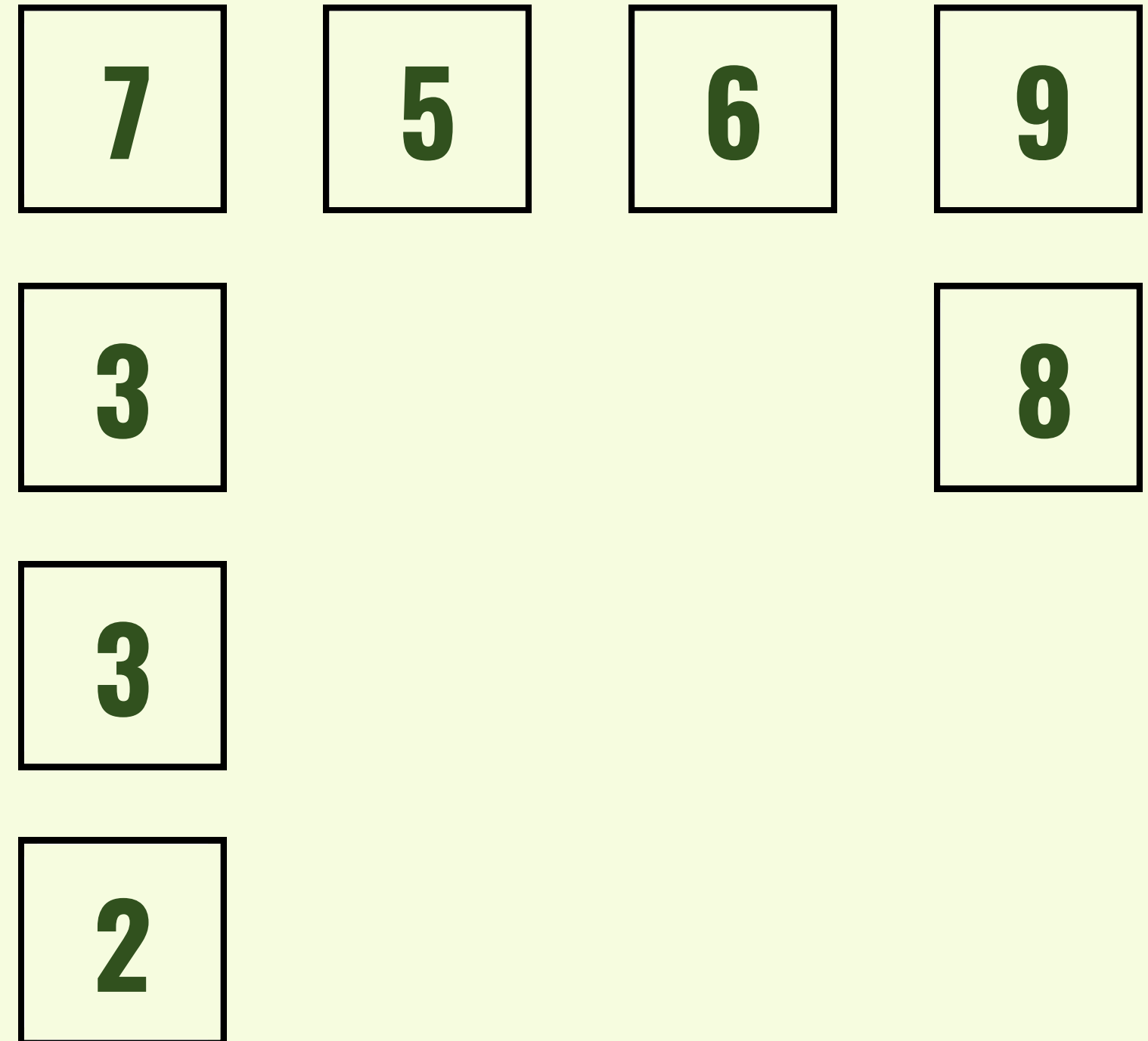
VETOR AUX:



O vetor aux não
armazena a maior
subsequência crescente.
O algoritmo falhou?

ESTRUTURA AUXILIAR

Vizualização
de “pilhas”:



ANÁLISE DO RESULTADO

O vetor aux de fato não representa a LIS existente no array, mas carrega informações suficientes para determinarmos a LIS, note que:

1. Obrigatoriamente elementos de uma mesma “pilha” não podem simultaneamente compor a LIS, pelos critérios definidos – portanto a LIS deve ter tamanho menor que ou igual a quantidade de pilhas;
2. Como a criação de pilhas novas só acontece quando encontramos um valor maior que todos as “tails” atuais das pilhas – então a partir da primeira pilha, existirá ao menos um caminho no qual os

Portanto o tamanho da LIS do array dado corresponde ao **tamanho do vetor aux;**

CÓDIGO E VEREDITO CSES

Code ▲

```
1 #include <bits/stdc++.h>
2
3 #define ll long long
4 #define vi vector<int>
5 #define vll vector<ll>
6 #define mp make_pair
7 #define pb push_back
8 #define endl '\n'
9 #define FASTIO ios::sync_with_stdio(false);cin.tie(0);cout.tie(0);
10
11 using namespace std;
12
13 int main()
14 {
15     FASTIO
16     int size; cin >> size;
17     vll array(size), aux;
18
19     for(int i = 0; i < size; i++)
20     {
21         cin >> array[i];
22         auto pos = lower_bound(aux.begin(), aux.end(), array[i]);
23         if(pos == aux.end())
24             aux.push_back(array[i]);
25         else
26             aux[pos-aux.begin()] = array[i];
27     }
28
29     cout << aux.size() << endl;
30
31     return 0;
32 }
```

Submission details

Task:	Increasing Subsequence
Sender:	tacopep
Submission time:	2026-06-25 18:27:03 +0300
Language:	C++ (C++17)
Status:	READY
Result:	ACCEPTED

Test results ▲

test	verdict	time	
#1	ACCEPTED	0.00 s	»
#2	ACCEPTED	0.00 s	»
#3	ACCEPTED	0.00 s	»
#4	ACCEPTED	0.00 s	»
#5	ACCEPTED	0.00 s	»
#6	ACCEPTED	0.02 s	»
#7	ACCEPTED	0.03 s	»
#8	ACCEPTED	0.03 s	»
#9	ACCEPTED	0.04 s	»
#10	ACCEPTED	0.04 s	»
#11	ACCEPTED	0.00 s	»
#12	ACCEPTED	0.00 s	»
#13	ACCEPTED	0.03 s	»
#14	ACCEPTED	0.00 s	»
#15	ACCEPTED	0.00 s	»
#16	ACCEPTED	0.00 s	»
#17	ACCEPTED	0.00 s	»
#18	ACCEPTED	0.02 s	»
#19	ACCEPTED	0.00 s	»

ANÁLISE DO ALGORITMO

Code ▲

```
1 #include <bits/stdc++.h>
2
3 #define ll long long
4 #define vi vector<int>
5 #define vll vector<ll>
6 #define mp make_pair
7 #define pb push_back
8 #define endl '\n'
9 #define FASTIO ios::sync_with_stdio(false);cin.tie(0);cout.tie(0);
10
11 using namespace std;
12
13 int main()
14 {
15     FASTIO
16     int size; cin >> size;
17     vll array(size), aux;
18
19     for(int i = 0; i < size; i++)
20     {
21         cin >> array[i];
22         auto pos = lower_bound(aux.begin(), aux.end(), array[i]);
23         if(pos == aux.end())
24             aux.push_back(array[i]);
25         else
26             aux[pos-aux.begin()] = array[i];
27     }
28
29     cout << aux.size() << endl;
30
31     return 0;
32 }
```

CUSTO DE TEMPO

Para cada um dos n elementos do array precisamos fazer uma busca binária e determinar sua posição no vetor aux

Complexidade: $O(n \log n)$

ANÁLISE DO ALGORITMO

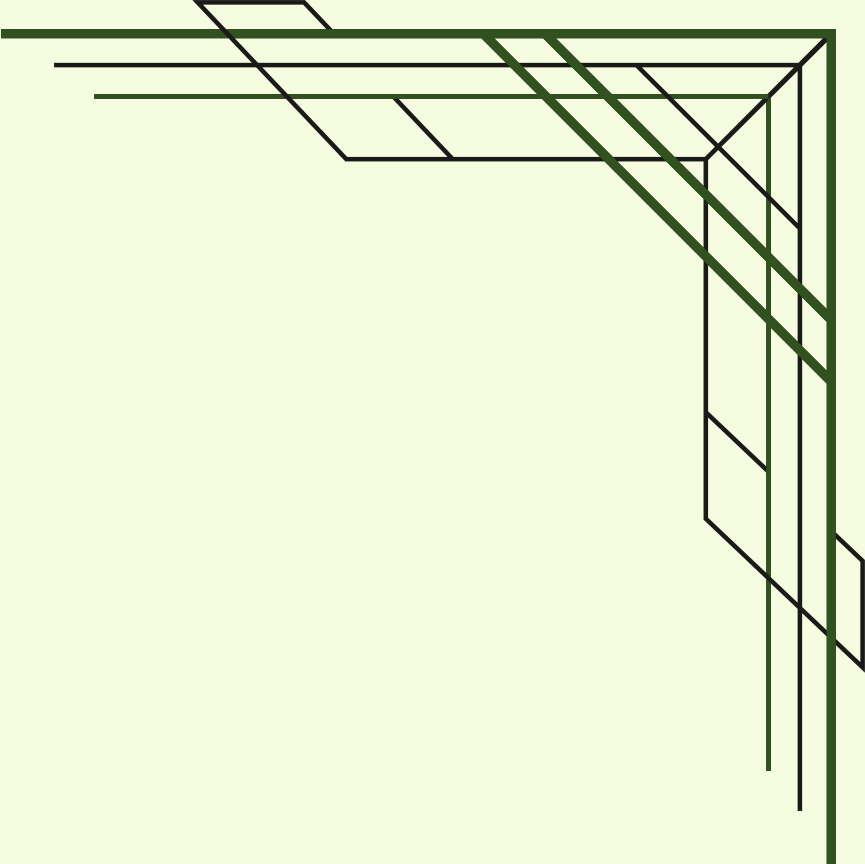
Code ▲

```
1 #include <bits/stdc++.h>
2
3 #define ll long long
4 #define vi vector<int>
5 #define vll vector<ll>
6 #define mp make_pair
7 #define pb push_back
8 #define endl '\n'
9 #define FASTIO ios::sync_with_stdio(false);cin.tie(0);cout.tie(0);
10
11 using namespace std;
12
13 int main()
14 {
15     FASTIO
16     int size; cin >> size;
17     vll array(size), aux;
18
19     for(int i = 0; i < size; i++)
20     {
21         cin >> array[i];
22         auto pos = lower_bound(aux.begin(), aux.end(), array[i]);
23         if(pos == aux.end())
24             aux.push_back(array[i]);
25         else
26             aux[pos-aux.begin()] = array[i];
27     }
28
29     cout << aux.size() << endl;
30
31     return 0;
32 }
```

CUSTO DE MEMÓRIA

Precisamos salvar na memória os elementos do array original e os elementos do array auxiliar no pior dos casos consumindo $2*n$ posições na memória.

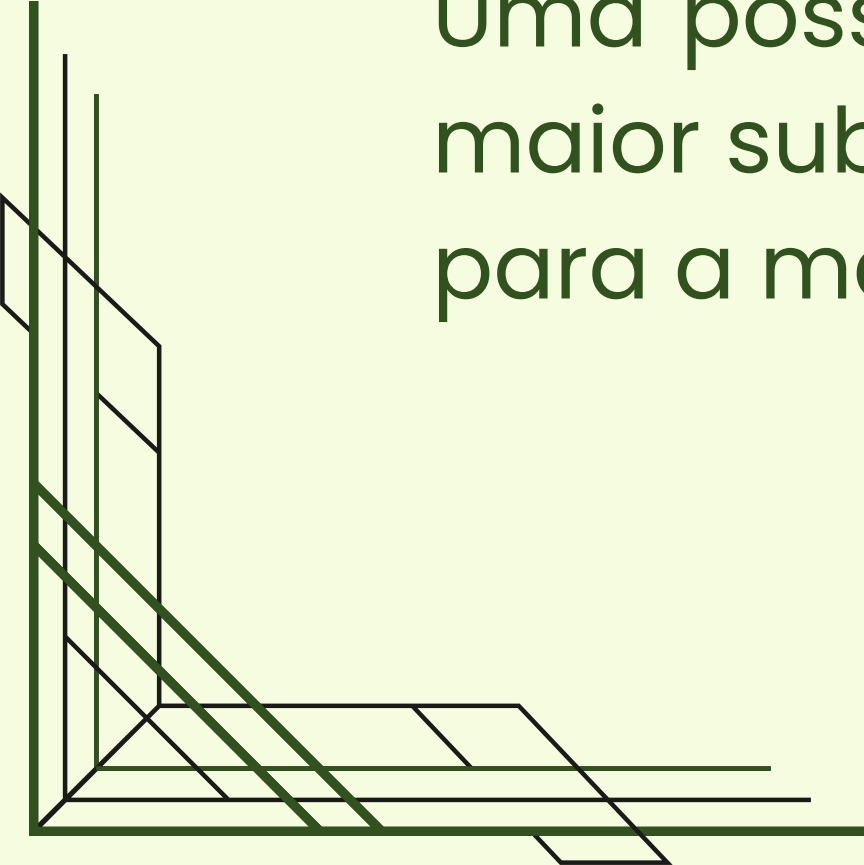
Complexidade: **$O(n)$**

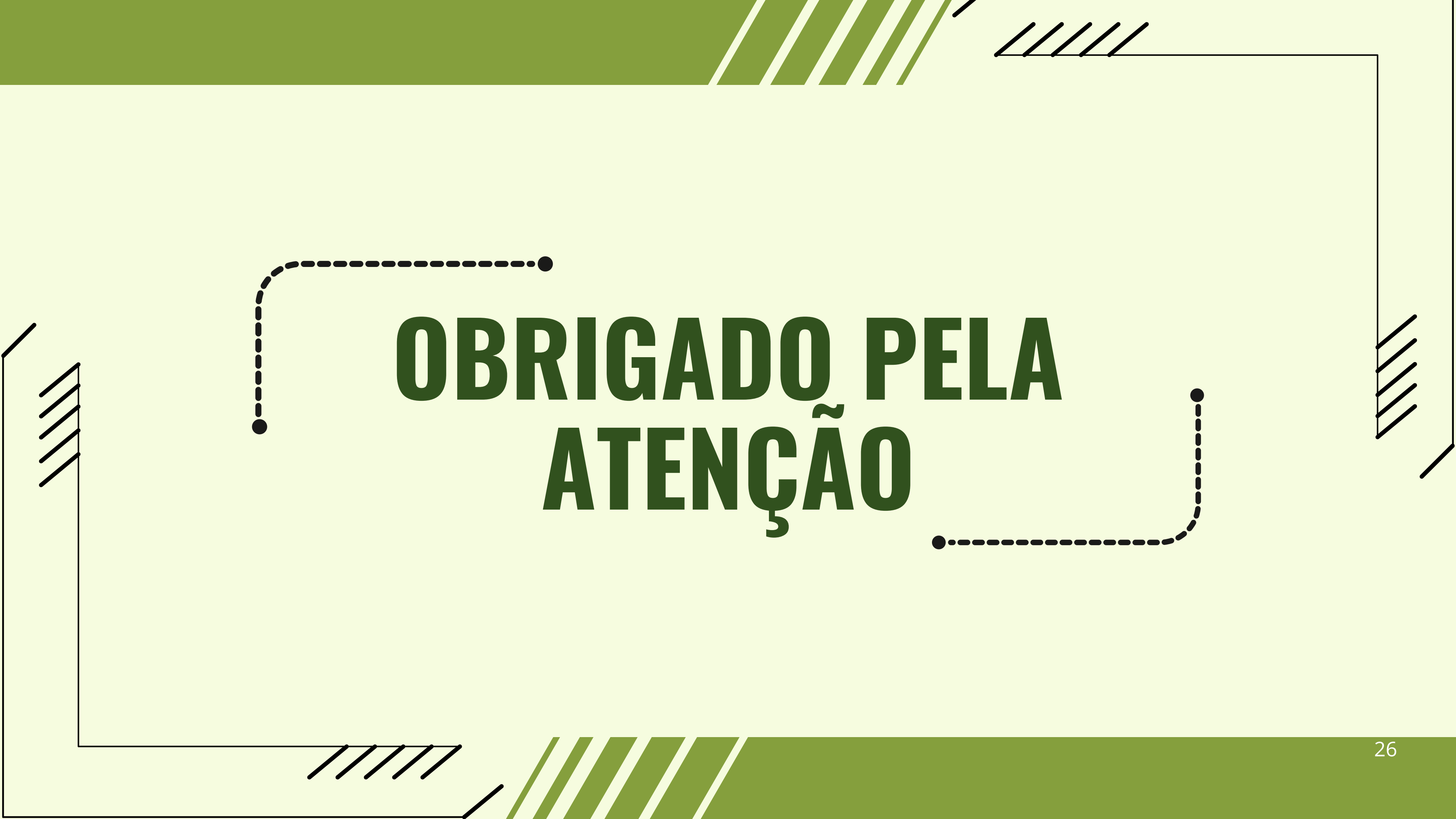


CONSIDERAÇÕES FINAIS

O problema é um exemplo simples e ilustrativo de como programação dinâmica permite várias abordagens únicas.

Uma possível extensão do problema seria pedir a impressão da maior subsequência crescente. Ou podemos reverter o problema para a maior subsequência decrescente (LDS).





**OBRIGADO PELA
ATENÇÃO**